

Taller de SQL y JDBC

Esta clase fue casi toda pizarrón: pasó gente a escribir código y SQL, y después de cada ejercicio soltó una tarea. Aquí está en el mismo orden — haces el ejercicio, y luego ves la tarea que dejó en ese punto. Son ocho ejercicios y cuatro tareas.

Los botones morados con ? explican los términos que el profesor usó dando por hecho que ya los conocías — de Bases de Datos, de POO, de materias anteriores. Ábrelos cuando algo no te suene; no interrumpen el ejercicio.

EJERCICIO 1 · MINUTO 06:28

Escribe la clase Carrera

Pidió un voluntario y le dio dos minutos para escribir la clase `Carrera` a partir del diagrama UML. No salió — y los errores que se cometieron son los que valen la pena revisar.

Su instrucción exacta fue: *"constructor con argumento cero, mutators y accessors, y al menos el toString. Escribame la clase."* Si esa frase no te dijo nada, no eres el único:

– ¿Qué son "mutators" y "accessors"?

Son los nombres formales de los **setters** y **getters**. Vienen de la teoría de POO y los vas a ver en libros y exámenes, pero casi nadie los usa al hablar.

- **Mutator** = método que *modifica* un atributo. En Java se llama `setAlgo(...)` y no devuelve nada (`void`).
- **Accessor** = método que *lee* un atributo. Se llama `getAlgo()`, no recibe nada y devuelve el valor.

Cuando pidió "mutators y accessors" estaba pidiendo los seis métodos: tres `set` y tres `get`, uno por atributo.

– ¿Qué es un "constructor con argumento cero"?

Un **constructor** es un método especial que se ejecuta al crear un objeto. Se llama igual que la clase y no declara tipo de retorno. Es donde el objeto queda listo para usarse.

"Con argumento cero" quiere decir que **no recibe nada** entre paréntesis — también se le dice constructor vacío o *no-arg*. Se ve así:

```
public Carrera() {  
}
```

Sirve para crear el objeto en blanco y llenarlo después con los setters. Java te regala uno gratis si no escribes ninguno — pero en cuanto escribes cualquier otro constructor, el gratis desaparece y tienes que ponerlo tú.

Alguien escribió esto en el pizarrón. Tiene **cinco errores**, todos los que realmente se discutieron en clase. Encuéntralos tocándolos:

EJERCICIO • CAZA EL ERROR

Cinco cosas están mal

Toca lo que creas incorrecto. Si le atinas se marca en verde; si no, se marca en rojo un momento.

0 de 5 encontrados

```
Class Carrera {  
.....  
  
Private int idCarrera;  
private final String nombreCarrera;  
private string descripcionCarrera;  
  
public Carrera ();  
.....  
  
public void setIdCarrera(int idCarrera) {  
this.idCarrera = idCarrera;  
}  
}
```

Cómo se lee el diagrama

Insistió en algo que conviene tener claro: **el diagrama de clases no tiene lenguaje**. Ese mismo dibujo puede ser TypeScript, C#, Python o Java — a ustedes les toca traducirlo. Aquí toca Java.

"Ahí están los planos de lo que hay que hacer. Literal, eso es lo que se les entrega a ustedes: el diagrama."

Y los símbolos de visibilidad del UML, que son los que hay que traducir a palabras de Java:

EN EL DIAGRAMA	EN JAVA	QUÉ SIGNIFICA
-	<code>private</code>	Solo dentro de la propia clase.
#	<code>protected</code>	La clase y quienes hereden de ella.
+	<code>public</code>	Cualquiera.
~	(sin modificador)	A nivel paquete. En Java se logra no escribiendo nada.

Otras cosas que aclaró al reconstruir la clase:

- El **constructor vacío sí lleva llaves**. No recibe argumentos, pero tiene cuerpo. Si no pones nada adentro está bien; de arranque ahí va un `super()` implícito.
- El `this` es obligatorio en los setters, porque el parámetro se llama igual que el atributo. Si el parámetro tuviera otro nombre —dijo `elIdCarrera` como ejemplo— podrías omitirlo.
- **CamelCase** es la notación que van a usar. Aclaró que no es la única (existe `snake_case`), pero es la de los libros formales de UML.
- Una clase así, con atributos privados y accesoros públicos, se llama **JavaBean** según el autor que revises.
- Mencionó que además de `toString()`, de `Object` se heredan `equals()`, `hashCode()`, `clone()` y `compareTo()`. Por ahora solo les pide el `toString()`.

— ¿Qué es ese `super()` que mencionó?

En Java **toda clase hereda de otra**. Si no dices de cuál, hereda de `Object`, que es la clase raíz de todo el lenguaje. Heredar significa que tu clase recibe gratis los métodos de la clase padre.

`super()` es la llamada al **constructor del padre**. Antes de construir tu objeto, Java construye primero la parte que heredó. Esa llamada la pone el compilador solo, invisible, en la primera línea de tu constructor — por eso dijo "de arranque tendría que ser un `super()`". No tienes que escribirla.

De ahí también sale el `toString()`: ya existe porque `Object` lo trae. Por eso "sobrescribirlo" tiene sentido — no lo estás inventando, lo estás reemplazando.

— ¿Qué es `@Override` y por qué lleva arroba?

Las cosas que empiezan con `@` en Java se llaman **anotaciones**. No son código que se ejecute: son etiquetas que le dejas al compilador o a un framework para que hagan algo especial con ese elemento.

`@Override` le dice al compilador: "este método está reemplazando uno que heredé". El compilador entonces **verifica que de verdad exista** en el padre. Si escribes `toString()` con un typo, en vez de crear silenciosamente un método nuevo que nadie va a llamar, te marca error.

Es opcional, pero ponerla te ahorra un bug muy difícil de ver. Vas a encontrarte muchas más anotaciones cuando lleguen a Spring.

– ¿Qué es CamelCase?

Es una **convención para nombrar** cosas cuando el nombre tiene varias palabras y no puedes usar espacios. Se pegan las palabras y cada una nueva empieza con mayúscula: `nombreCarrera`, `getIdCarrera`. Se llama así porque las mayúsculas hacen jorobas como las de un camello.

Hay dos variantes, y en Java se usan las dos:

- **lowerCamelCase** — empieza en minúscula. Para atributos y métodos: `idCarrera`, `setNombreCarrera`.
- **UpperCamelCase** o **PascalCase** — empieza en mayúscula. Para nombres de clases: `Carrera`, `CarreraDAO`.

La alternativa que mencionó es **snake_case**, que separa con guion bajo: `nombre_carrera`. Es lo normal en bases de datos y en Python, pero no en Java.

– ¿Qué es un "JavaBean"?

Es simplemente **el nombre que se le da a una clase con esta forma exacta**: atributos privados, un constructor vacío, y getters y setters públicos para cada atributo. Nada más. No es una tecnología ni una librería que instales.

Importa porque muchas herramientas de Java **dan por hecho que tus clases son así**. Cuando lleguen a Hibernate o a Spring, esas herramientas van a crear tus objetos llamando al constructor vacío y luego llenarlos llamando a los setters. Si tu clase no sigue el patrón, no funcionan.

Por eso insistió tanto en el constructor vacío: no es capricho, es el contrato que las herramientas esperan.

– ¿Por qué la clase tiene que ser `public` para ejecutarse?

Dijo: "para que se pueda ejecutar sí o sí debe de haber una clase pública". La razón es que `public` significa **visible desde cualquier parte del programa**. Si una clase no es pública, solo la ven las clases de su mismo paquete (su misma carpeta).

Para que Java pueda arrancar tu programa necesita poder *ver* la clase desde afuera. Además, el archivo debe llamarse exactamente igual que la clase

pública: `Carrera.java` para `public class Carrera`. Eso sí es obligatorio y el compilador lo revisa.

TAREA 1 minuto 15:17

Reporte de lectura — "Lectura rápida Java"

*"Primera tarea del día: reporte de lectura, lectura rápida Java, **porque no lo han leído**. Entonces léanlo, en serio léanlo."*

Ojo con esto: en la clase pasada solo dijo "léanlo". Ahora es un **reporte de lectura** — o sea, ya es un entregable, no nada más leerlo. Son 58 páginas y están en la plataforma.

EJERCICIO 2 · MINUTO 25:35

Qué va y qué no va en un DTO

Antes de mandarlos a investigarlo, explicó la idea con la tabla que ya tienen.

Un DTO tiene la **misma forma** que la clase de entidad: clase pública, atributos privados, getters y setters. La diferencia es qué campos lleva.

Su pregunta al grupo fue directa: **para insertar una carrera, ¿qué necesitas realmente?** Y un alumno contestó bien — el nombre y la descripción. El `idCarrera` no, porque lo genera la base de datos con `auto_increment`.

— ¿Qué significan `auto_increment`, `primary key`, `not null` y `unique` ?

Son **restricciones**: reglas que le pones a una columna al crear la tabla, y que la base de datos hace cumplir sola. Si algo las viola, rechaza la operación.

- `primary key` — **llave primaria**. La columna que identifica de forma única cada renglón. No se puede repetir ni quedar vacía. Es contra la que filtras cuando quieres tocar exactamente un registro.
- `auto_increment` — **la base lleva la cuenta sola**. Cada renglón nuevo recibe el siguiente número: 1, 2, 3... Por eso no mandas el id al insertar: si lo mandas, estorbas. Esta palabra es exclusiva de MySQL.
- `not null` — **obligatorio**. Ese campo no puede quedar vacío.
- `unique` — **no se puede repetir**. En la tabla, dos carreras no pueden llamarse igual. Distinto de la llave primaria: puedes tener varias columnas `unique`, pero llave primaria solo una.

Y `null` **no es cero ni cadena vacía**: significa "aquí no hay dato". Es la diferencia entre una calificación de 0 y una materia que aún no se califica.

— ¿Y por qué dijo que debería ser `text` en vez de `varchar` ?

Los dos guardan texto; cambia cuánto y cómo.

- `varchar(150)` — texto de **longitud variable con tope**. Guarda hasta 150 caracteres y ocupa solo lo que uses. Bueno para nombres, correos, títulos.
- `text` — para textos **largos, sin un tope razonable de antemano**.
Descripciones, comentarios, artículos.

Su comentario fue que, formalmente en MySQL, una descripción larga va en `text`. Pero aclaró que da igual para lo que están haciendo: **del lado de Java las dos llegan como `String`**. La distinción importa en la base, no en el código.

Luego complicó el ejemplo. Imagina que la tabla tuviera además una columna `activo`, que acepta nulos y tiene valor por defecto `false` a nivel base de datos. Su caso real: la carrera ISISA de ESCOM estuvo inactiva unos años y se reactivó hace poco.

EJERCICIO · SELECCIONA

Arma el DTO de inserción

La tabla `Carrera` tiene cuatro columnas. ¿Cuáles van en el DTO que usarás **para insertar** una carrera nueva?

- ☐ **idCarrera**
int, auto_increment, primary key
- ☐ **nombreCarrera**
varchar(150), not null, unique
- ☐ **descripcionCarrera**
varchar(500), not null
- ☐ **activo**
acepta nulos, valor por defecto false en la base de datos

Revisar

Reiniciar

El segundo ejemplo que dio es el que más aclara la idea, porque ahí el DTO no corresponde a ninguna tabla. Quiere una pantalla que muestre cuántos alumnos hay inscritos por carrera:

ISC 2020	100
ISC 2009	50
IA	80
LCD	30

Esa consulta toca varias tablas — `Alumno` , `Carrera` , y probablemente una de inscripción— pero **el resultado que quieres pintar solo tiene dos columnas**: nombre y cantidad. Esa estructura de dos campos es un DTO. No es una tabla, no es necesariamente una vista: es el molde de lo que vas a mostrar.

— ¿Qué es una "vista"?

Una **vista** (*view*) es una consulta guardada dentro de la base de datos a la que le pones nombre, y después la usas como si fuera una tabla.

Si escribes todos los días una consulta larga que junta tres tablas, la guardas como vista `alumnos_por_carrera` y luego consultas `SELECT * FROM alumnos_por_carrera` . La vista no guarda datos propios: cada vez que la consultas, por debajo se ejecuta la consulta original.

Por eso dijo "¿es una vista? No necesariamente, pero puede ser". El DTO vive **en tu código Java**; la vista vive **en la base de datos**. Pueden tener la misma forma sin ser lo mismo.

Y de paso soltó una advertencia sobre modelado: si metieran todas las carreras en una sola tabla con un campo de texto repetido, "le diría que tachezote para su base de datos. Esa cosa no está normalizada."

— ¿Qué es "normalizar" una base de datos?

Es **organizar las tablas para que ningún dato se repita**. Cada hecho se guarda en un solo lugar, y las demás tablas lo referencian con un id.

Su ejemplo malo sería guardar el nombre de la carrera dentro de cada alumno:

```

Alumno
-----
id  | nombre | carrera
1   | Ana    | Ingenieria en Sistemas Computacionales
2   | Luis   | Ingenieria en Sistemas Computacionales
3   | Sam    | Ingeniera en Sistemas Computacionales  <-- typo

```

Tres problemas concretos, y por eso "tachezote":

- El texto largo se repite en cada renglón: **desperdicia espacio**.

- Si la carrera cambia de nombre, hay que corregir **miles de renglones** y si se te escapa uno, quedan inconsistentes.
- Basta un dedazo para que "la misma" carrera aparezca como dos distintas — mira el renglón 3. **Contar alumnos por carrera deja de dar bien.**

La versión normalizada: el nombre vive una sola vez en la tabla `Carrera`, y el alumno guarda nada más el `idCarrera`. Corriges el nombre en un lugar y se corrige para todos.

Lo de "primera forma normal" que mencionó es el primer nivel de un conjunto de reglas (hay varias formas normales, 1FN, 2FN, 3FN...). Eso es tema de Bases de Datos — y es justo lo que te va a pedir en el modelo entidad-relación del proyecto, así que vale la pena repasarlo.

TAREA 2 minuto 25:04

Investigación — patrón Transfer Object (DTO)

"¿Qué es, para qué sirve, con qué se come y cómo se adereza el patrón de diseño Transfer Object?"

Esas cuatro preguntas son literalmente el formato que pide — las usó igual para el DAO. Dijo que no lo iba a escribir en el pizarrón a propósito: "les toca a ustedes investigarlo".

TEORÍA CORTA · MINUTO 33:41

Las tres formas de hablarle a la base

Lo mínimo que necesitas antes de los ejercicios de SQL.

CLASE	VEREDICTO	POR QUÉ
Statement	Nunca	Se arma concatenando texto, como en PHP. Si no sanitizas todo, "les meten gol" — inyección SQL. Dijo textual: "existe, pero no la usen".
PreparedStatement	Esta	Es la que van a usar. Recibe los valores como parámetros, no concatenados.
CallableStatement	Después	Optimizada para stored procedures y funciones. Requiere que alguien haga de DBA y te dé los nombres y argumentos. Aclaró que el cambio entre una y otra "tampoco es tan complejo".

Lo que hacen las dos últimas, en sus palabras: toman la consulta, la **analizan**, la **desmenuzan**, **generan un plan de ejecución**, determinan qué argumentos esperan —de qué tipo y de qué longitud— y solo aceptan esos valores. Por eso no se les puede colar SQL disfrazado de dato.

– "Que les metan gol" – ¿qué es la inyección SQL?

Es la vulnerabilidad más clásica que existe, y entenderla de verdad toma treinta segundos. El problema aparece cuando **armas la consulta pegando texto**:

```
String sql = "SELECT * FROM Usuario WHERE nombre = '" + nombre + "'"
```

Si el usuario escribe `Ana`, la consulta queda bien. Pero el usuario no tiene por qué escribir un nombre. Si escribe esto:

```
' OR '1'='1
```

la consulta que llega a la base es:

```
SELECT * FROM Usuario WHERE nombre = '' OR '1'='1'
```

Y como `'1'='1'` siempre es verdad, **devuelve la tabla completa**. Lo que el usuario escribió dejó de ser un dato y se convirtió en parte de la instrucción. Con un `DELETE` en vez de un `SELECT`, se lleva todo.

Por qué `PreparedStatement` lo impide: manda la consulta y los valores *por separado*. Primero le dice a la base "esta es la instrucción, con dos huecos", la base la compila así, y solo después le pasa los valores. Cuando el valor llega, la instrucción **ya está armada y cerrada** — el texto solo puede ser dato. Aunque escribas `' OR '1'='1'`, la base lo busca literalmente como un nombre y no encuentra a nadie.

Ese es todo el motivo de los `?`. No es estilo: es la diferencia entre una app segura y una que regala su base de datos.

– ¿Qué es un "stored procedure" y qué es un DBA?

Un **stored procedure** (procedimiento almacenado) es código SQL **guardado dentro de la base de datos**, con nombre y parámetros, como si fuera una función. En vez de mandar la consulta desde Java, mandas "ejecuta el procedimiento `insertarCarrera` con estos dos valores".

Ventaja: la lógica vive en un solo lugar y la base la tiene precompilada.

Desventaja: parte de tu programa deja de estar en tu código, y queda amarrada a ese manejador.

DBA es *Database Administrator*: la persona encargada de la base de datos — su diseño, permisos, respaldos y procedimientos almacenados. En una empresa es alguien distinto del programador. Por eso dijo que para `CallableStatement` "necesitan sí o sí de la existencia de un DBA": hay que preguntarle cómo se llaman los procedimientos y qué argumentos reciben, porque tú no los escribiste.

Y remató: "para fines académicos les toca a ustedes jugar el rol de DBA" — o sea, en su proyecto ustedes hacen los dos papeles.

También aclaró por qué el SQL del pizarrón es específicamente de MySQL:

`AUTO_INCREMENT` no existe en los demás manejadores. En PostgreSQL sería el tipo `SERIAL` o una secuencia; en SQL Server, un entero con atributo de identidad.

TAREA 3 minuto 32:16

Investigación — patrón Data Access Object (DAO)

"¿Qué es, para qué sirve, cómo se come y cómo se adereza el patrón de diseño Data Access Object?"

Adelantó lo que vas a encontrar, así que te sirve de guía: el patrón se describe con **problemática, intención y estructura**. Y en estructura pide **una interfaz** (el *qué* se va a hacer) más **una implementación** (el *cómo*). Sirve para desacoplar el acceso a datos: en teoría cambias de MySQL a PostgreSQL sin tocar el resto. Sobre eso fue honesto — "al menos eso es lo que dice la teoría; llevarlo a la práctica... en fin".

TAREA 4 minuto 33:14

Reporte de lectura — "Lectura rápida JDBC"

"¿Qué creen? Va a haber una cuarta tarea. Otro reporte de lectura: lectura rápida JDBC."

Una alumna reclamó que ya la había dicho, y aclaró la diferencia: **la primera es de Java, por el tema de construcción de clases; esta es de JDBC**, por lo que sigue. Son dos reportes distintos.

EJERCICIOS 3 A 7 • MINUTO 42:35

El CRUD en el pizarrón

Aquí pasaron cinco personas, una por operación. Escribe tú el SQL — se valida de verdad, no es opción múltiple. Ignora mayúsculas y el punto y coma final si quieres.

Todo se hace contra la tabla que ya conocen:

```
create table Carrera (  
  idCarrera          int not null auto_increment primary key,  
  nombreCarrera      varchar(150) not null unique,  
  descripcionCarrera varchar(500) not null  
);
```

EJERCICIO 3 · CREATE

Inserta tu carrera

Su instrucción fue: "**insértenme la carrera a la que ustedes pertenecen**". Usa el nombre `ISC` y la descripción `Carrera del futuro`, que es lo que se escribió en clase.

INSERT INTO ...

Revisar

EJERCICIO 4 · UPDATE

Corrige la descripción

El registro tiene `idCarrera = 1`. Cambia la descripción de `Carrera del futuro` a `Ingeniería en Sistemas Computacionales`. **Aquí es donde el compañero se equivocó** — a ver si tú caes en lo mismo.

UPDATE ...

Revisar

EJERCICIO 5 · DELETE

Borra el registro

Borrado **físico**, del registro con `idCarrera = 1`. Piensa bien contra qué columna lo haces.

DELETE ...

Revisar

– Dijo "es un borrado físico, ¿eh?" — ¿hay de otro tipo?

Sí, y la distinción importa mucho en sistemas reales.

- **Borrado físico** — `DELETE FROM`. El renglón desaparece de la tabla. No hay deshacer. Es lo que pidió en el ejercicio.
- **Borrado lógico** — el renglón se queda, pero lo marcas como inactivo:
`UPDATE Carrera SET activo = false WHERE idCarrera = 1`. Para el usuario desapareció; para la base sigue ahí.

Casi todo sistema serio usa el lógico, por tres razones: puedes deshacer un error, conservas el historial (¿qué pasa con los alumnos inscritos a una carrera que borraste?), y muchas veces la ley obliga a conservar los datos.

Y aquí se cierra un círculo: ¿te acuerdas del atributo `activo` del ejemplo del DTO, el de la carrera ISISA que estuvo inactiva? Esa columna es el mecanismo del borrado lógico. ISISA nunca se borró de la base — se marcó inactiva y años después se reactivó. Con un borrado físico, esa información se habría perdido.

EJERCICIO 6 · READ ONE

Consulta una sola carrera

Todos los datos de la carrera ISC, pero **la consulta hazla por id** — así lo pidió, para que no fuera lo mismo que traer todo.

```
SELECT ...
```

Revisar

EJERCICIO 7 · READ ALL

Trae todos los registros

Este lo escribió él mismo en el pizarrón "para que vean que estoy de buenas". Es el más corto de todos.

```
SELECT ...
```

Revisar

Eso que acabas de escribir tiene nombre

Las cinco operaciones juntas son un **CRUD funcional**: Create, Read, Update, Delete. La R son los dos SELECT — según el autor los vas a ver como

`readAll` / `readOne`, o `retrieveAll`.

Y lo que va más allá del CRUD, dijo, son los **finders**: métodos de búsqueda que ejecutan una consulta por algún criterio, como buscar por fragmento del nombre.

Repasar Bases de Datos

"Bueno, no sé si preocuparme... o no. No va a ser tarea: repasen Bases de Datos."

Lo dijo después de ver cuánto batallaron con SQL básico. No lo va a calificar, pero el mensaje es evidente. Va con algo que aclaró antes: **esta materia es integradora** — asume que ya pasaron Bases de Datos, Análisis y Diseño de Sistemas, Ingeniería de Software, Paradigmas de Programación y Tecnologías para la Web. Y cerró con "si no, no se apuren, vamos aprendiendo; sobre la marcha va a ir saliendo, así es que no se me desanimen".

EJERCICIO 8 · MINUTO 60:44

Ahora el mismo SQL, desde Java

La última media hora la dedicó a empezar la clase `CarreraDAO`. Alcanzó a llegar hasta el método de conexión.

El primer cambio es que el SQL deja de tener valores escritos y pasa a tener **signos de interrogación**:

```
private static final String SQL_INSERT =  
    "INSERT INTO Carrera (nombreCarrera, descripcionCarrera) VALUES (?, ?)";
```

"¿Qué valores vamos a insertar? No sé, no me importa, no me interesa: es bronca del usuario. No me interesa si viene de una aplicación de escritorio, una web, una móvil o la consola."

Y aprovechó para un detalle de Java que vale la pena: **en Java no hay constantes**. La palabra `const` está reservada pero no se puede usar — es un recuerdo de C. Lo que se hace en su lugar es declarar el atributo `final`.

— Entonces, ¿qué hace cada palabra de `private static final` ?

Son tres modificadores distintos apilados, y cada uno decide una cosa aparte:

- `private` — **quién lo ve**. Solo esta clase. Nadie de afuera necesita tu SQL.
- `static` — **a quién pertenece**. Sin `static`, cada objeto que crees tiene su propia copia. Con `static`, existe **una sola copia compartida por toda la clase**. Si creas cien `CarreraDAO`, el texto del SQL se guarda una vez, no cien.
- `final` — **si se puede cambiar**. Una vez asignado, no se puede reasignar. Si alguien intenta modificarlo, el error salta al compilar, no en producción.

Juntos dicen: "un texto fijo, privado, que existe una sola vez para toda la clase". Eso es lo más cercano a una constante que tiene Java, y por convención esas se escriben en **MAYÚSCULAS con guiones bajos** — por eso `SQL_INSERT` y no `sqlInsert`.

`static` es el que más cuesta al principio. La regla corta: **sin `static` pertenece al objeto; con `static` pertenece a la clase.**

La cadena de conexión

Desarmó la URL pieza por pieza. Ármala tú:

EJERCICIO · ARMA LA CADENA

Construye la URL de conexión

Base de datos `MisCursos`, en tu propia máquina, MySQL en su puerto por defecto. Toca las piezas en orden.

Toca las piezas de abajo, en orden...

`mysql://`

`jdbc:`

`localhost`

`:3306`

`/MisCursos`

Reiniciar

— ¿Qué son `localhost`, `127.0.0.1` y el `3306`?

`localhost` y `127.0.0.1` son la misma cosa: tu propia máquina.

`127.0.0.1` es la dirección IP reservada que siempre apunta a ti mismo, y

`localhost` es su nombre. Cuando la base de datos corre en tu laptop, ahí es donde la buscas.

Si la base estuviera en otra máquina, ahí va su IP — como el `192.168.1.1` que usó en el pizarrón para dibujar la base en un equipo y la aplicación en otro.

El `3306` es el puerto. Una máquina puede tener muchos programas escuchando a la vez, así que cada uno se aparta un número. Si la IP es el edificio, el puerto es el número de departamento. 3306 es el que MySQL usa por convención — igual que 80 es el de la web. Como es el de siempre, dijo que "si lo ponen bien y si no también".

Ese mismo 3306 es el que apareció en la clase pasada al mapear puertos de contenedores. Es el mismo concepto.

Si tu instalación lo necesita, se le pueden agregar parámetros: **el primero con `?` y los siguientes con `&`**, siempre en pares nombre-valor. Zona

horaria y codificación son los típicos.

El método completo, como lo dejó

```
private Connection conexion;

private void obtenerConexion() {

    String usuario = "root";
    String clave   = "tu_password";
    String urlBD   = "jdbc:mysql://localhost:3306/MisCursos";
    String driverMySQL = "com.mysql.cj.jdbc.Driver";

    try {
        Class.forName(driverMySQL);
        conexion = DriverManager.getConnection(urlBD, usuario, clave);
    } catch (ClassNotFoundException | SQLException e) {
        e.printStackTrace();
    }
}
```

– ¿Qué es un "driver" y por qué hay que descargarlo?

Un **driver** es una librería que actúa de traductor. Java sabe hablar JDBC —un lenguaje genérico de bases de datos— pero **no sabe hablar MySQL**. El driver de MySQL traduce entre los dos.

Por eso hay un driver distinto por cada manejador: uno para MySQL, otro para PostgreSQL, otro para Oracle. Tu código Java no cambia; cambias el traductor. **Esa es exactamente la promesa del patrón DAO** del que te dejó la investigación.

Es un archivo `.jar` que hay que agregar al proyecto. Si falta, sale `No suitable driver found`.

– ¿Qué son Maven y Gradle? ¿Y qué es un `.jar` ?

Un `.jar` (Java ARchive) es un **paquete comprimido con código Java ya compilado**. Puede ser una librería que usas, o tu aplicación entera lista para ejecutarse. Cuando dije "mi archivito `.jar`", mi ejecutable, que voy a estar echando a andar en cualquier otra PC", se refería a tu programa empaquetado.

Maven y Gradle son gestores de dependencias. En vez de que tú busques el `.jar` del driver en internet, lo descargues y lo copies a mano, declaras en un archivo de texto qué necesitas y ellos lo bajan solos, con la versión correcta y con lo que esa librería a su vez necesite.

Por eso dijo "Maven, Gradle y todas esas cosas ya lo solucionan: nada más ponen la referencia y va y lo descarga". En Maven ese archivo se llama `pom.xml`.

— ¿Qué hace `Class.forName()` exactamente?

Carga una clase a **partir de su nombre escrito como texto**, mientras el programa ya está corriendo. Normalmente escribes el nombre de la clase en el código y el compilador la resuelve; aquí el nombre es una cadena que podrías leer de un archivo de configuración.

Eso se llama **reflexión**: código que manipula clases en tiempo de ejecución en vez de en tiempo de compilación.

Por qué se usaba aquí: al cargarse, el driver se registra solo con `DriverManager` — le avisa "yo sé hablar MySQL". Sin ese registro, `DriverManager` no sabría a quién pedirle la conexión.

Y por qué ya no hace falta: desde JDBC 4 los drivers se anuncian automáticamente. Por eso dijo "vamos a pensar que esto ya no es necesario, pero vamos a pensar que es obligatorio" — lo escribió por formalismo y porque lo vas a ver en códigos viejos.

— ¿Qué es `DriverManager` ?

Es la clase de Java que **lleva la lista de los drivers disponibles** y te entrega conexiones. Cuando le pides `getConnection(url, usuario, clave)`, mira la URL, ve que empieza con `jdbc:mysql:`, busca en su lista quién sabe hablar MySQL, y le encarga a ese driver crear la conexión.

Tú nunca creas una `Connection` con `new`: siempre se la pides a `DriverManager`. Es lo que te permite no depender de la clase concreta del driver.

Los puntos que remarcó de este método:

- Usa `java.sql.Connection`, no la del driver de MySQL. El driver trae su propia clase específica y funciona — pero si la usas, rompes la promesa del patrón de poder cambiar de manejador. Por eso van los paquetes genéricos.
- El usuario no debería ser `root`. Lo formal es crear un usuario por base de datos con los privilegios mínimos. Para fines académicos aceptó root.
- La D de `Driver` va en mayúscula — es una clase. Y aclaró que registrarlo con `Class.forName()` ya no es necesario desde hace tiempo, pero lo escribió por formalismo.
- Las dos excepciones son distintas. `ClassNotFoundException` es que no encontró el archivo del driver. `SQLException` cubre todo lo demás:

usuario mal, contraseña mal, o cualquier error en la cadena de conexión — nombre de base equivocado, puerto, IP, un punto y coma donde iban dos puntos, una diagonal en vez de dos.

- El `printStackTrace()` no lo hagas. Sus palabras: "este cachito no lo hagan, pero sirve". Lo correcto es mandarlo a un log — archivo, servidor, log del sistema — no vaciar la pila completa en pantalla.
- Debería devolver algo. Lo dejó en `void` por tiempo, pero señaló que en teoría debe regresar si pudo conectarse o no: un booleano, o un entero.

— ¿Qué es un "stack trace" y por qué no debe imprimirse?

Cuando algo truena, Java no solo dice qué falló: dice **toda la cadena de llamadas que llevó hasta ahí**. El método que falló, quién lo llamó, quién llamó a ese, hasta el inicio del programa. Esa lista es el *stack trace*, y se ve como treinta renglones de texto.

Es información valiosísima para depurar. El problema es **a dónde la mandas**. `printStackTrace()` la escribe en la consola, y eso tiene dos fallas: en un servidor la consola no la ve nadie y se pierde, y le muestra a quien sea las rutas internas y estructura de tu aplicación — información útil para alguien que quiera atacarte.

Lo correcto es mandarlo a un **log**: un archivo donde el programa va anotando lo que pasa, con fecha y nivel de gravedad. Ahí queda guardado, lo puedes revisar después, y el usuario nunca lo ve.

De ahí la broma del SAES y el "Null ADODB Exception" en rojo: eso es justo un error interno escapándose a la pantalla del usuario.

De paso, una definición que puede caer en examen: "En Java no hay errores, hay excepciones." Matizó que la clase `Error` sí existe — pero cuando aparece un `Error` "es porque algo tronamos con ganas".

— Entonces, ¿cuál es la diferencia entre `Error` y `Exception`?

En Java las dos cosas que pueden "tronar" cuelgan de una clase común llamada `Throwable`, y se parten en dos ramas:

- **`Exception`** — problemas que tu programa **puede y debe manejar**. Que la base no responda, que el archivo no exista, que el usuario meta letras donde iban números. Se atrapan con `try/catch` y el programa sigue vivo.
- **`Error`** — fallas de la máquina virtual, de las que **no tiene sentido intentar recuperarse**. Se acabó la memoria, se desbordó la pila por una recursión infinita. Ahí no hay nada que hacer. Eso es "tronar con ganas".

Hay una segunda división que te va a importar más en la práctica:

- **`Checked`** — el compilador **te obliga** a atraparlas o declararlas. `SQLException` y `ClassNotFoundException` son de estas. Por eso el

`try/catch` del método no es opcional: sin él, el código no compila. Java te obliga porque son cosas que razonablemente van a pasar.

- **Unchecked** — nadie te obliga. La más famosa es `NullPointerException`, cuando usas un objeto que resultó ser `null`. Se consideran errores de programación, no situaciones esperables.

Y el `|` del `catch (ClassNotFoundException | SQLException e)` significa "cualquiera de las dos, trátalas igual". Se llama *multi-catch* y evita escribir dos bloques idénticos.

CIERRE

Las cuatro tareas, juntas

Se despidió con "nos vemos el próximo lunes, tienen un montón de tareas". Aquí están las cuatro para ir marcándolas.

0 / 5

☐ Tarea 1 • Reporte de lectura: "Lectura rápida Java"

58 páginas, en la plataforma. Ya no basta con leerlo — ahora es un reporte entregable.

☐ Tarea 2 • Investigación: patrón Transfer Object (DTO)

Qué es, para qué sirve, con qué se come y cómo se adereza. No lo escribió en el pizarrón a propósito.

☐ Tarea 3 • Investigación: patrón Data Access Object (DAO)

Mismas cuatro preguntas. Adelantó que lo vas a encontrar descrito como problemática, intención y estructura — y que la estructura pide una interfaz más una implementación.

☐ Tarea 4 • Reporte de lectura: "Lectura rápida JDBC"

Distinta de la de Java: esta es por el tema de la conexión a base de datos. Unas 60 páginas.

☐ Repasar Bases de Datos (no calificable)

Lo dijo después de ver cómo batallaron con el CRUD en el pizarrón. No es entregable, pero el mensaje fue claro.

Nota sobre las dos lecturas: son las mismas que mencionó en la clase 2, pero cambiaron de categoría. Antes solo pidió leerlas; ahora son **reportes**

de lectura, o sea entregables con formato. Si ya las tenías pendientes, ahora pesan más.